

ENGINEERING DECISION LOG

AI Customer Support Agent — Amazon Help on Twitter

Purpose

This log records the principal engineering choices made while building the support-agent prototype. Each entry captures the decision, the rationale, and the practical consequence for the system or evaluation. The goal is to make the project reproducible and show how constraints, observed data, and evaluation evidence influenced the design.

01

Brand selection

Amazon Help was selected from the dataset because it provides high message volume and a broad range of realistic support scenarios.

Why it mattered: This gives the taxonomy enough diversity to cover delivery, billing, account, product, and support-frustration cases without building a brand-specific system around a narrow issue set.

02

Conversation-pair construction

Amazon replies were joined back to the customer tweets using the response relationship instead of treating tweets as independent records.

Why it mattered: The task is fundamentally about customer → support interactions, so paired examples provide both the incoming problem and the historical resolution used later for retrieval.

03

Dataset subsampling

The full approximately 3M-tweet dataset was not processed end-to-end. A representative working subset was used, including a 15,000-message retrieval corpus.

Why it mattered: The take-home requires a working, evaluable system. Subsampling keeps experimentation fast and reproducible while retaining enough historical diversity for semantic retrieval.

Engineering Decisions

04

English-language filtering

An ASCII-ratio heuristic (>0.9 ASCII characters) was chosen as a lightweight preprocessing filter.

Why it mattered: It was fast and simple to implement, but it was explicitly treated as a heuristic rather than a language detector. Later analysis confirmed that Latin-script European languages could pass through.

05

Initial intent taxonomy

The taxonomy began with ten intent categories derived from sampled Amazon Help messages.

Why it mattered: Starting with a compact taxonomy avoids unnecessary fragmentation and keeps classification practical for a support-routing system.

06

Taxonomy evolution during labeling

Three additional categories — `technical_issue`, `follow_up_acknowledgment`, and `non_english` — were added after real golden-set examples exposed gaps.

Why it mattered: Rather than forcing real customer messages into poorly fitting labels, the taxonomy was allowed to evolve from observed data. This improved semantic coverage.

Engineering Decisions

07

Separate escalation signal

Escalation was modeled as a separate boolean flag in addition to the customer-service-escalation intent.

Why it mattered: Intent answers “what is the customer asking about?” while escalation answers “should a human handle this?” A refund can therefore be both refund_return and an escalation.

08

Hosted LLM selection

NVIDIA NIM was selected to provide hosted LLM inference without adding paid API costs, using an OpenAI-compatible client.

Why it mattered: The project needed repeated model calls during experimentation, so a free hosted endpoint was practical for the take-home constraint.

09

Model iteration

Models were changed during development as availability, latency, and output behavior were tested. The final workflow used the available Nemotron family for classification/generation/escalation tasks as appropriate.

Why it mattered: Model selection was treated as an engineering constraint rather than assuming one model was optimal. End-of-life and latency issues encountered during development directly influenced the final choice.

Engineering Decisions

10

Deterministic classification

The intent classifier uses a zero-temperature prompt containing the full allowed label list and asks for exactly one label.

Why it mattered: Classification is an evaluation-sensitive task. Restricting the output space and using temperature zero reduces unnecessary variability and makes results easier to reproduce.

11

Output post-processing

Raw model outputs were cleaned before scoring by stripping leaked thinking markers, whitespace artifacts, and mapping recognized outputs back to the allowed label set.

Why it mattered: Hosted models occasionally returned malformed text. Evaluation should measure classification behavior rather than fail simply because a valid label contains formatting noise.

12

Semantic retrieval design

Historical customer messages were embedded with all-MiniLM-L6-v2 and compared using cosine similarity. The top three similar cases were used as grounding examples.

Why it mattered: A lightweight embedding + in-memory cosine search was sufficient for a 15,000-example corpus and avoided unnecessary vector-database infrastructure for a take-home project.

Engineering Decisions

13

Retrieval-grounded generation

The reply generator receives similar historical customer messages and the actual Amazon replies associated with them.

Why it mattered: Grounding the model in observed support behavior makes the response style and resolution approach more evidence-based than generating a generic support response from scratch.

14

Layered escalation policy

High-risk intents and obvious repeated-frustration signals are handled by deterministic rules; remaining cases are passed to the LLM.

Why it mattered: This reduces unnecessary LLM calls, provides predictable handling for sensitive cases, and reserves model judgment for ambiguous situations.

15

Two-baseline evaluation

The system was compared against a trivial most-common-class baseline and a simple keyword-matching baseline.

Why it mattered: The trivial baseline establishes the minimum bar, while keyword matching represents a simple heuristic solution. The LLM should demonstrate value beyond both rather than being evaluated in isolation.

Decision Summary

The decisions fall into five themes:

Data discipline. The project uses paired conversations, deliberate subsampling, and an explicit acknowledgment of the limitations of the ASCII-based language filter.

Taxonomy from evidence. The intent set was not treated as fixed before seeing the data. New categories were introduced when real examples showed that the initial taxonomy was insufficient.

Pragmatic model selection. Hosted inference was chosen for cost and iteration speed, while model changes were driven by actual availability, latency, and output behavior.

Grounded generation and layered risk control. Replies use historical Amazon responses as retrieval evidence, while escalation combines deterministic rules with model judgment.

Evaluation over demos. Two simple baselines were included, model outputs were cleaned before scoring, and failure modes are treated as first-class results rather than hidden.

Evaluation status

Development evaluation currently reports 54.64% intent accuracy and 0.45 macro F1, versus 28.21% for the trivial baseline and 19.29% for the keyword baseline. Escalation accuracy is 69.42%, with 0.79 precision and 0.54 recall for true escalations. For final submission, the official golden evaluation should be fixed to 250 examples to match the assignment's 150–250 requirement.

Key engineering lesson

The strongest evidence from the prototype is not that the model is universally accurate. It is that the complete pipeline can be measured, compared against simple alternatives, and inspected at the failure-mode level. The next iteration should target the highest-cost weaknesses — product/content classification and missed escalations — and prove improvement with a cleaner evaluation.